

NIFTY-50 Investment Intelligence Platform

Technical Report

Tharun Rathod (23115034)

Akhil Ravuri (23115121)

Dataset: Kaggle rohanrao/nifty50-stock-market-data

Date: June 2026

1. Introduction

We built an investment intelligence platform using the NIFTY-50 stock market dataset from Kaggle. The dataset has daily OHLCV data for 50 stocks from January 2000 to April 2021, around 235K rows total. The main idea behind our approach is that stock markets go through different phases like bull runs, crashes, and sideways periods. A model that works in a bull market might completely fail during a crash. So instead of building one model and hoping it works everywhere, we first detect what kind of market we are in using a Hidden Markov Model, and then adapt our predictions, portfolios, and risk estimates accordingly.

The platform covers all three mandatory tasks (stock prediction, portfolio construction, risk assessment) and also implements optional tasks including explainability, anomaly detection, investment scoring, and deployment as a Streamlit web app.

2. Exploratory Data Analysis

We started by loading all 52 CSV files from the dataset. Two files (stock_metadata.csv and NIFTY50_all.csv) were skipped because they don't have the same Date column format. After loading, we had about 235K rows covering 65 unique stock symbols.

2.1 Price Trends

We plotted closing prices for representative stocks across sectors. Stocks like RELIANCE and BAJFINANCE showed strong growth especially after 2015. IT stocks like TCS and INFY had more steady growth with smaller drawdowns. Banking stocks moved together most of the time.

2.2 Return Distributions

Daily returns are not normally distributed. We saw fat tails on both sides and slight negative skew for most stocks. This means extreme drops happen more often than a normal distribution would predict. This is why we used VaR and regime based risk management instead of just relying on standard deviation.

2.3 Correlation Analysis

Banking stocks (HDFCBANK, ICICIBANK, SBIN, AXISBANK) are highly correlated with each other, correlation around 0.7 to 0.85. IT stocks have moderate correlation with financials. FMCG stocks like HINDUNILVR and ITC have lower correlation with the rest. This helped us understand which stocks actually add diversification to a portfolio.

2.4 Volume Patterns

We computed volume-price correlation for each stock. Some stocks showed negative correlation which can indicate accumulation patterns. Volume tends to spike during market crashes and during breakout moves.

3. Feature Engineering

We computed 34 features for each stock from the raw OHLCV data. All features are standard technical indicators used in quantitative finance.

Features computed:

- Daily returns and log returns
- Moving averages: MA20, MA50, MA200

- Exponential moving averages: EMA12, EMA26 (used for MACD)
- MACD line, signal line, and histogram
- RSI with 14 day period
- Bollinger Bands with 20 day window and 2 standard deviations
- 30 day rolling volatility from log returns
- Average True Range (ATR) with 14 day period
- Lag features: closing price lagged by 1, 2, 3, 5, and 10 days

We did not use any external data, APIs, or sentiment data. Everything is derived from the provided OHLCV columns only.

4. Market Regime Detection

This is the core idea of our project. We used a Gaussian Hidden Markov Model with 3 hidden states to classify the market into Bull, Bear, and Sideways regimes.

4.1 How it works

First we computed an equal weighted index return by averaging daily returns across all stocks for each day. Then we fit a 3 state GaussianHMM from the hmmlearn library on these returns. The model learns three distributions corresponding to three market states. We label them by sorting the state means: lowest mean = Bear, middle = Sideways, highest = Bull.

4.2 Results

The model correctly identified known market events without any manual labels. The 2008 Global Financial Crisis shows up as a Bear regime. The 2003 to 2007 period is labeled Bull. The COVID crash in March 2020 is detected as Bear. The 2016 to 2017 period after demonetization shows a mix of Sideways and Bull states.

We also computed a transition matrix showing how likely it is to move from one regime to another. Bull to Bull transitions are the most persistent. Bear regimes tend to be short but intense.

4.3 Why this matters

The detected regime is fed as a numeric feature to the XGBoost model. Portfolios are also adapted based on the current regime. During Bear regimes, the system switches to defensive sectors like FMCG, Pharma, and IT. During Bull regimes, it increases exposure to momentum stocks.

5. Stock Prediction Models

5.1 Baseline: Linear Regression

We used a simple Linear Regression as our baseline model. Input features are Close_lag1, Close_lag2, Close_lag3, MA20, and Volume. Target is the closing price. This gives us a reference point to compare against more complex models.

5.2 Main Model: XGBoost

Our main model is XGBoost with the following configuration: 500 trees, max depth 6, learning rate 0.05, subsample 0.8, colsample_bytree 0.8. It uses 16 features including all technical indicators plus the regime label as a numeric input (Bull=2, Sideways=1, Bear=0).

5.3 Train/Test Split

We used a time based split where the last 20% of data is the test set. We never shuffle the data because that would leak future information into training. This is important for financial time series.

5.4 Evaluation

Results on RELIANCE test set:

Model	RMSE	MAE	R2	Dir. Accuracy
Linear Regression	42.15	31.28	0.9891	52.3%
XGBoost + Regime	15.87	11.42	0.9953	84.9%

XGBoost significantly outperforms the baseline on all metrics. The R2 of 0.9953 means the model explains most of the variance. Directional accuracy of 84.9% means it correctly predicts whether the price goes up or down about 85% of the time on the test set.

We also used SHAP (SHapley Additive exPlanations) to understand feature importance. Close_lag1 is the most important feature, followed by MA20 and RSI. The regime feature contributes meaningfully, confirming that regime awareness helps prediction.

6. Portfolio Construction

We built three portfolio strategies for different investor profiles. All portfolios are constructed from the available NIFTY-50 stocks using their historical daily returns.

6.1 Conservative Portfolio

Uses inverse volatility weighting. We pick the lowest volatility 25% of stocks, filter for Sharpe > 1.0, and weight them inversely to their volatility. Max 10% per stock and 30% per sector. This gives a portfolio focused on capital preservation.

6.2 Balanced Portfolio

Uses mean variance optimization with `scipy.optimize.minimize`. We maximize the Sharpe ratio subject to constraints: weights sum to 1, no short selling, max 15% per stock, 30% per sector. Candidates are filtered by volatility (below 75th percentile) and Sharpe (above 0.5).

6.3 Aggressive Portfolio

Uses momentum weighting based on 6 month cumulative returns. We pick the top 25% performers by cumulative return and weight them proportionally. Max 20% per stock. This portfolio chases recent winners and has higher concentration.

6.4 Regime Conditional Portfolios

We also built separate portfolios for each regime. During Bear regimes, the system allocates to defensive sectors (FMCG, Pharma, IT) using inverse volatility weights. During Bull regimes, it uses momentum weights with higher concentration. During Sideways, it optimizes for Sharpe ratio. This is the key differentiator of our approach.

7. Risk Assessment

We compute the following risk metrics for individual stocks and portfolios:

- Annualized Volatility: daily standard deviation multiplied by $\sqrt{252}$
- Sharpe Ratio: (annualized return minus 6% risk free rate) divided by annualized volatility
- Sortino Ratio: same as Sharpe but uses only downside deviation
- Maximum Drawdown: largest peak to trough decline in cumulative returns
- Value at Risk (95%): the 5th percentile of daily returns, represents worst expected daily loss
- Beta: covariance of stock returns with index returns divided by index variance

We display these in a color coded table in the app. Green shading for high Sharpe, red for deep drawdowns. This makes it easy to compare risk across stocks at a glance.

7.1 Anomaly Detection

We flag days where a stock's daily return exceeds 3 standard deviations from its mean. These are plotted on the price chart as red dots. Most anomalies cluster around the 2008 crisis and March 2020, which makes sense since those were extreme market events.

8. Investment Score

We created a composite investment score from 0 to 100 that combines five dimensions:

Component	Weight	What it measures
Return Score	30%	CAGR and recent 6 month return
Risk Score	25%	Volatility, drawdown, VaR (inverted, lower risk = higher score)
Momentum Score	20%	RSI, MACD histogram, price vs MA50 and MA200
Stability Score	15%	Positive month ratio and return consistency
Volume Score	10%	Recent vs average volume ratio, volume-price correlation

Each component is normalized to 0-100 across all stocks, then combined with the weights above.

Grades: A (80+), B (65+), C (50+), D (35+), F (below 35).

The score gives a quick way to rank stocks across multiple dimensions instead of looking at just one metric like returns or volatility alone.

9. Explainability

Every recommendation the system generates is backed by computed metrics. The recommendation engine takes the current regime, RSI, 90 day Sharpe ratio, volatility, and recent returns as inputs and generates a recommendation (BUY, HOLD, ACCUMULATE, or HOLD with caution) with specific numbers.

We also have a sector rotation table that shows annualized returns for each sector under each regime. This helps understand which sectors perform best in different market conditions. For example, IT and Financials tend to do better in Bull regimes while FMCG and Pharma hold up better in Bear regimes.

For the prediction model, SHAP values show which features contributed most to each prediction. This lets users understand why the model predicted a certain price instead of treating it as a black box.

10. Deployment

The platform is deployed as a Streamlit web application on Streamlit Community Cloud. It has 6 tabs covering EDA, Regime Detection, Stock Prediction, Portfolio Construction, Risk Assessment, and Explainability.

The app uses @st.cache_data on the three heavy computations (data loading, feature engineering, HMM fitting). On the first load the app takes about 30 to 40 seconds on Streamlit Cloud free tier because it reads 52 CSV files (235K rows), computes features for every stock, and fits the HMM. After that first load everything is cached and tab switches are instant. Only the Run Predictions button triggers fresh computation since XGBoost training is done on demand per stock.

The code is on GitHub at github.com/tharun99856/NIFTY-50-Investment. To run locally: clone the repo, install requirements.txt, put the CSV files in data/raw/, and run streamlit run app.py.

11. Limitations

- The dataset ends in April 2021 so we cannot validate on recent market data
- We only used the provided Kaggle dataset, no live data or external sources

- XGBoost prediction accuracy drops in Bear regimes because crash behavior is harder to predict
- The HMM is fit on the full dataset, in production you would retrain periodically
- Portfolio optimization assumes historical patterns continue, which is not guaranteed
- No transaction costs or slippage are modeled in portfolio backtests

12. Conclusion

We built a complete investment intelligence platform that goes beyond simple price prediction. The regime detection approach is the main contribution. It lets us adapt everything (predictions, portfolios, risk estimates) to the current market state instead of using a single static model. The HMM correctly identifies known market events like the 2008 crisis and COVID crash without any manual labels. XGBoost with the regime feature achieves strong results on the test set. The three portfolio strategies cover different risk profiles, and the regime conditional portfolios show how allocation can be adapted dynamically.

All code is open source and reproducible. The Streamlit app makes the analysis accessible without needing to run any code manually.